

BOOTCAMP FRONTEND & REACT · BLOQUE REACT · CLASE 7

La sesión se vuelve seria

La sesión sobrevive al F5, se valida con el servidor y expira sola. Rutas protegidas. Variables de entorno. Y la tienda en producción.

4 h 15 · L·M·V

Retomamos donde cortamos

STEERCL

SIN REPASOS LARGOS: RETOMAMOS Y SEGUIMOS

El plan de hoy

0. Reapertura

8'

el punto de corte + los planes de los alumnos

A. Remate de la clase 6

0-75'

solo lo pendiente · guion de la clase 6

B. La sesión completa

80'

guardar · validar con /auth/me · expirar ·
RutaProtegida · .env**C. No repintar de más**

60'

medir primero · memo · useMemo · lazy

D. Producción

35'

vercel.json · despliegue en Vercel · URL pública

F. Cierre + 5 tareas

20'

push demostrado, dudas y anticipo

 **Receso de 15 min** alrededor de las 2:00 · verde = **el corazón del día** · se corta a las 3:50 donde toque

TRES COSAS QUE UNA SESIÓN TIENE QUE HACER

La sesión que sobrevive

1 • GUARDAR

el usuario se guarda en el disco (`almacen.ts`) y se lee al arrancar **comprobando su forma**: token de texto, vencimiento numérico. Si no, no hay sesión.

2 • VALIDAR

un token guardado puede ser viejo o estar anulado. Quien decide es el servidor: GET `/auth/me` con el token en el header `Authorization: Bearer.`

3 • EXPIRAR

guardamos **cuándo** vence (`expiraEn`) y un `setTimeout` cierra la sesión en ese instante. La limpieza del efecto lo cancela.

```
useEffect(() => {  
  if (!usuario) return;  
  // 1. si ya venció → cerrar · 2. obtenerPerfil(token) → si el servidor lo rechaza,  
  cerrar  
  // 3. setTimeout(cerrar, expiraEn - Date.now())  
  return () => { controlador.abort(); clearTimeout(temporizador); };  
}, [usuario]);
```

LA PREGUNTA DE LA TAREA: ¿QUIÉN VIGILA /CHECKOUT?

RutaProtegida: una ruta padre que revisa la sesión



```

<Route element={<RutaProtegida />}} // ruta padre sin path
  <Route path="/checkout" element={<Checkout />}} /> // toda ruta de adentro queda
  protegida
</Route>
  
```

Dos detalles: replace reemplaza la entrada del historial (el botón atrás no rebota entre login y checkout) · state={{ desde }} viaja con la navegación sin aparecer en la URL: el login devuelve al usuario a donde iba.

PARA RECONOCERLOS SIN SUFRIRLOS — NINGUNO SE CODEA

Errores frecuentes de sesión II

1 · guardar el token sin validarlo nunca

que esté en el disco no garantiza nada; un token que el servidor ya anuló seguiría "logueado" para siempre → `/auth/me` al arrancar

2 · confiar solo en `expiraEn`

sirve para cerrar a tiempo en el navegador, pero quien decide si el token vale es el servidor → las dos comprobaciones juntas

3 · Navigate sin replace

el historial queda con una entrada de más y el botón atrás rebota entre el login y la página protegida

4 · subir el `.env` a GitHub

una clave privada publicada ya no sirve, hay que cambiarla. `.gitignore` primero, commit después; `.env.example` sí se sube

PRIMERO SE MIDE, DESPUÉS SE OPTIMIZA

Render **no es** pintar

Re-renderizar = React ejecuta la función del componente y **compara** el resultado con el anterior. Pintar = tocar el DOM, solo donde hubo diferencia. Re-renderizar es barato; React ya minimiza lo caro. Entonces, ¿cuándo optimizar? En tres casos concretos:

memo(Componente)

un componente se re-renderiza porque su PADRE cambió, aunque sus props sean las mismas. memo lo salta si cada prop es el mismo valor u objeto. No bloquea los cambios de Context.

useMemo(fn, deps)

un cálculo pesado se repite en cada render con los mismos datos. Recalcula solo si cambió algo del array; el resultado sigue siendo un valor calculado, no estado.

lazy + Suspense

código que el usuario tal vez nunca visite se descarga igual. `lazy(() => import(...))` lo parte en un archivo aparte; Suspense muestra algo mientras llega.

Se optimiza lo que se midió. React DevTools → "Highlight updates" muestra qué se re-renderiza. Con 24 productos nada de esto se nota: se aprende aquí para usarlo donde sí hace falta.

PARA RECONOCERLOS SIN SUFRIRLOS — NINGUNO SE CODEA

Errores frecuentes de optimización

1 • optimizar sin medir

memo en todo "por si acaso" suma comparaciones que cuestan más de lo que ahorran. Primero Highlight updates, después la herramienta

2 • useMemo con dependencias incompletas

usar adentro una variable que no está en el array devuelve resultados viejos. El linter lo marca: hazle caso

3 • lazy sin Suspense

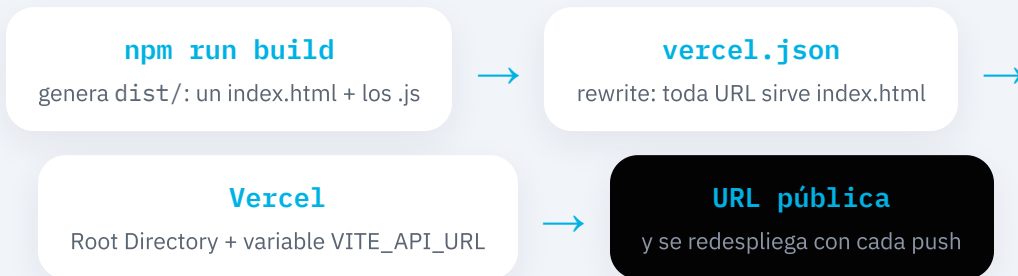
React no sabe qué mostrar mientras descarga el archivo y muestra un error que lo dice claramente

4 • lazy en TODO

partir la tienda en cuarenta archivos hace que cada clic descargue algo. Se parte por rutas que no todos visitan (Checkout, Login), no Home

LA TIENDA EN INTERNET, CON DIRECCIÓN PROPIA

Producción en Vercel



```
// vercel.json - el servidor no conoce tus rutas: /producto/7 no es un archivo
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

Variables de entorno: solo llegan al navegador las que empiezan con `VITE_`; se leen con `import.meta.env`; en producción se configuran en Vercel, no en el código. El `.env` no se sube; el `.env.example` sí.

NO SE INSTALA NADA: TODO DESDE VERCEL.COM CON TU CUENTA DE GITHUB

Vercel **paso a paso**

1. Crear la cuenta

vercel.com → **Sign Up** → plan **Hobby** (gratis) → **Continue with GitHub** → Authorize

2. Nuevo proyecto

Dashboard → **Add New...** → **Project**

3. Importar el repo

Import Git Repository → buscar tu repo → **Import** (si no aparece: Adjust GitHub App Permissions)

4. Root Directory

Edit → clase-01 → **tienda-react** → Continue.
Framework Preset: **Vite**

5. Variable de entorno

Environment Variables → Key VITE_API_URL · Value
`https://dummyjson.com` → **Add**

6. Deploy

Deploy → Build Logs → Congratulations → **Continue to Dashboard** → Domains → tu URL .vercel.app

Después: cada **git push** a main vuelve a desplegar solo (pestaña **Deployments**). Los nombres de los botones pueden variar; el orden no.

PARA RECONOCERLOS SIN SUFRIRLOS — NINGUNO SE PROVOCA

Errores frecuentes al desplegar

1 • el build falla en Vercel pero no en tu PC

Linux distingue mayúsculas en los nombres de archivo; Windows no. El import debe coincidir LETRA POR LETRA con el archivo (`./layout/layout` ↔ `layout.tsx`). El registro dice cuál

2 • "No package.json found"

faltó el Root Directory: el proyecto está en `clase-01/tienda-react`, no en la raíz → Settings → General → Root Directory → Redeploy

3 • abre pero no carga productos

falta `VITE_API_URL`: Settings → Environment Variables → Add → Deployments → ... → **Redeploy** (las variables se leen al construir, no en vivo)

4 • 404 al recargar `/producto/7`

falta `vercel.json`, o quedó dentro de `src` en vez de la raíz del proyecto, junto a `package.json`

HASTA DONDE LLEGAMOS HOY

Lo de hoy

sesión completa

se guarda comprobando los datos, se valida con /auth/me (Authorization: Bearer) y expira sola con un temporizador

RutaProtegida

ruta padre que protege todo lo de adentro: Outlet si hay sesión, Navigate con replace y state si no

variables de entorno

VITE_, import.meta.env, .env fuera del repo y .env.example adentro

render no es pintar

memo evita el re-render por el padre, useMemo el recálculo, lazy la descarga — siempre después de medir

rewrite de SPA

el servidor no conoce tus rutas: index.html para todo

URL pública

la tienda en Vercel, con despliegue automático en cada push

CRITERIO DE ENTREGA: BUILD SIN ERRORES + PUSH + LA URL DE VERCEL EN EL README

5 tareas

FÁCIL

Replica la clase, hasta donde llegamos, y pega la URL pública de tu tienda al inicio del README. Es LA tarea.

INTERMEDIO

Proteger /pedidos: la página entra dentro de RutaProtegida y el enlace "Mis pedidos" solo aparece con sesión iniciada.

INTERMEDIO

El nombre de la tienda como variable: VITE_NOMBRE_TIENDA en .env y .env.example, tipada, usada en el Header y el título; configurada también en Vercel.

DIFÍCIL

Skeleton como fallback por ruta: Detalle con lazy y un Suspense propio cuyo fallback sea una silueta del detalle. ¿Se puede anidar Suspense?

DIFÍCIL

"Recordarme": checkbox en el login; apagado → sesión en sessionStorage. El ÚNICO archivo que debería cambiar es `almacen.ts`.

Lo que sigue

El proyecto final, con todo lo aprendido

componentes · estado · efectos · API · rutas · formularios

Context · hooks propios · sesión con JWT · rendimiento · producción

// tema libre (no tiene que ser una tienda) · arquitectura hexagonal = punto extra

La próxima clase: la rúbrica en pantalla, elegir tema y armar la base del proyecto desde cero con las cuatro carpetas. **Trae tres ideas de proyecto** y la URL de tu tienda en el README.

Trae las tareas hechas y el build en cero. Nos vemos.